

KubeQuest — Guide de survie pour la keynote (français)

Fichier local, **non commité dans git**. Date : 2026-07-09. Principe : **presque tout se pilote depuis node1 avec `kubectl`**. node1 est le control-plane et gère tout le cluster.

0. Aide-mémoire express (le plus utilisé)

But	Commande
Se connecter à node1	<code>ssh node1</code>
Voir tous les pods	<code>ssh node1 kubectl get pods -A</code>
Voir les nœuds	<code>ssh node1 kubectl get nodes -o wide</code>
Accéder à l'app	<code>ssh -L 8080:localhost:8080 node1 'kubectl -n laravel-app prod port-forward svc/laravel-app 8080:8080' → http://localhost:8080</code>
Accéder à Grafana	<code>ssh -L 3000:localhost:3000 node1 'kubectl -n monitoring port-forward svc/kube-prometheus-grafana 3000:8080' → http://localhost:3000</code>
Accéder à ArgoCD	<code>ssh -L 8081:localhost:8081 node1 'kubectl -n argocd port-forward svc/argocd-server 8081:8080' → http://localhost:8081</code>

1. Les quatre nœuds

Nœud	Rôle	IP publique (change, sauf node1)	IP privée (stable)	Instance ID
node1	control-plane	35.156.165.130 (Elastic, fixe)	10.2.50.23	i-02dfc53b800341c20
node2	worker	63.180.201.231 (change)	10.2.50.190	i-0635cb49fcfc65063
node3	ingress	18.196.141.176 (change)	10.2.50.90	i-0fcdcb2880d2d6fb3
node4	monitoring	3.67.67.127 (change)	10.2.50.113	i-05c12cfa80ed7e5e3

- Région `eu-central-1`. Les VM **s'éteignent chaque nuit** → avant la keynote, **démarrer les 4** dans la console AWS et vérifier les IP publiques.
- L'app `laravel-app` tourne sur **node2** ; la stack monitoring sur **node4** ; les contrôleurs ingress sur node2/node4.

2. Connexion aux nœuds

2.1 node1 (seul nœud joignable directement avec la clé `ec2-epitech`)

```
ssh node1
# équivaut à : ssh -i ~/.ssh/ec2-epitech ec2-user@35.156.165.130
```

Le `~/.ssh/config` contient déjà les alias `node1 / node2 / node3 / node4` (clé `ec2-epitech`).

2.2 node2 / node3 / node4 (désormais joignables directement)

✓ Votre clé publique `ec2-epitech` a été injectée dans l' `authorized_keys` de node2/3/4 : les **4 nœuds se connectent maintenant comme node1** : `bash ssh node2 # worker (10.2.50.190) ssh node3 # ingress (10.2.50.90) ssh node4 # monitoring (10.2.50.113)` ⚠ node2/3/4 **changent d'IP publique au redémarrage** : mettre alors à jour le `HostName` dans `~/.ssh/config` (actuel : node2=63.180.201.231, node3=18.196.141.176, node4=3.67.67.127 ; node1 est en Elastic IP, fixe). Nouvelle IP : console AWS EC2, ou `ssh node1 kubectl get nodes -o wide` (les IP privées ne changent pas).

Si la connexion directe échoue (IP changée sans mise à jour du config), deux options de secours :

Secours A — clé d'origine kubequest.pem (connecte les 4 nœuds) : Récupérer `/kubequest/group-50/ssh-private-key` dans AWS Systems Manager → Parameter Store (bouton Show), la sauver en `~/.ssh/kubequest.pem`, corriger les droits (`icacIs`), puis :

```
ssh -i ~/.ssh/kubequest.pem ec2-user@63.180.201.231 # node2 (IP publique actuelle)
ssh -i ~/.ssh/kubequest.pem ec2-user@18.196.141.176 # node3
ssh -i ~/.ssh/kubequest.pem ec2-user@3.67.67.127 # node4
```

C'est ainsi que le projet accédait aux nœuds à l'origine (cf. `scripts/windows/install-k8s.ps1` : `ssh -i kubequest.pem ec2-user@<ip>`).

Secours B — sans SSH, ouvrir un shell sur n'importe quel nœud depuis node1 :

```
ssh node1
kubectl debug node/ip-10-2-50-190.eu-central-1.compute.internal -it --image=busybox
# ^ lance un pod temporaire avec accès à l'hôte. Nom du nœud via 'kubectl get nodes'.
# Faire 'chroot /host' pour agir sur la machine. Le pod est supprimé à la sortie.
```

2.3 Une IP publique a changé

node1 est en Elastic IP (fixe). node2/3/4 changent au redémarrage : voir la nouvelle IP dans la console EC2. **La gestion quotidienne ne passe que par node1**, donc aucun impact.

3. Accès aux interfaces web (application / ArgoCD / monitoring)

⚠ Important : l'ingress **n'est pas exposé publiquement** (rien n'écoute sur :80/443, les NodePorts 32222/32214 sont bloqués par le security group). On accède donc **toujours via « tunnel SSH + kubectl port-forward »** — robuste et insensible aux changements d'IP. Chaque commande : la lancer dans un terminal et **le laisser ouvert**, puis ouvrir l'URL localhost dans le navigateur. Ctrl+C pour fermer.

3.1 Le site web (application Laravel)

```
ssh -L 8080:localhost:8080 node1 'kubectl -n laravel-app-prod port-forward svc/laravel-app 8080:80'
```

→ <http://localhost:8080> (page compteur, le bouton +1 incrémente).

3.2 ArgoCD (GitOps)

```
ssh -L 8081:localhost:8081 node1 'kubectl -n argocd port-forward svc/argocd-server 8081:80'
```

→ <http://localhost:8081> , login **admin** / mot de passe **4SUlul0BxohZNskt** (si expiré : `ssh node1 "kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d"`)

3.3 Grafana (dashboards monitoring)

```
ssh -L 3000:localhost:3000 node1 'kubectl -n monitoring port-forward svc/kube-prometheus-grafana 3000:80'
```

→ <http://localhost:3000> , login **admin** / mot de passe **3d5vp6v7xYxx9IL5bFTLFGTxTwINpLpM1bwSnxHd** Contient : dashboards Kubernetes préconfigurés (source Prometheus) + **source de données Loki** (Explore → Loki pour les logs).

3.4 Prometheus (métriques, protégé par SSO)

```
ssh -L 9090:localhost:9090 node1 'kubectl -n monitoring port-forward svc/kube-prometheus-kube-prometheus 9090:9090'
```

→ <http://localhost:9090> (accès direct au service = contourne le SSO). Pour démontrer le **login SSO** : passer par l'ingress (\$4.7), compte **admin@kuberequest.local** / **password**.

3.5 Kubernetes Dashboard

```
ssh -L 8443:localhost:8443 node1 'kubectl -n kubernetes-dashboard port-forward svc/kubernetes-dashboard 8443:443'
```

→ <https://localhost:8443> (auto-signé, cliquer « continuer »). Login par Token :

```
ssh node1 kubectl -n kubernetes-dashboard create token admin-user --duration=8h
```

4. Tests / démonstrations en direct (tout après `ssh node1`)

4.1 Vue d'ensemble

```
kubectl get nodes -o wide      # 4 nœuds Ready
kubectl get pods -A           # tout Running, 0 erreur
```

4.2 Application (multi-réplicas + persistance)

```
kubectl -n laravel-app-prod get pods      # 2/2 Running
kubectl -n laravel get pvc                # data-mysql-0 + mysql-backup-pvc = Bound
# 200 depuis un namespace autorisé par la NetworkPolicy :
kubectl -n ingress-nginx run t --rm -i --restart=Never --image=curlimages/curl -- \
  curl -s -o /dev/null -w "%{http_code}\n" http://laravel-app.laravel-app-prod.svc.cluster.local
```

4.3 Auto-scaling HPA (d  mo de charge)

```
kubectl get hpa -A                                # laravel 3-10, ingress 2-5
# G  n  rer de la charge (autre terminal) :
kubectl -n ingress-nginx run load --rm -i --restart=Never --image=curlimages/curl -- \
  sh -c 'while true; do wget -q -O- http://laravel-app.laravel-app-prod.svc.cluster.local >/dev/nul
l; done'
# Observer la mont  e en charge :
watch kubectl get hpa,pods -n laravel-app-prod
```

4.4 Validating webhook OPA (Gatekeeper) — d  mo de refus

```
# Pod SANS le label app.kubernetes.io/name -> REFUS   :
kubectl -n laravel-app-prod run bad --image=nginx --restart=Never
# attendu : Error ... denied the request ... label obligatoire manquant
# Avec le label -> ACCEPT   :
kubectl -n laravel-app-prod run good --image=nginx --restart=Never --labels=app.kubernetes.io/name=
demo
kubectl -n laravel-app-prod delete pod good
kubectl get constraint                                # pods-must-have-name-label
```

4.5 NetworkPolicy — d  mo d'isolation

```
kubectl -n laravel-app-prod get netpol
# Depuis un namespace non autoris   -> timeout (000) :
kubectl -n default run x --rm -i --restart=Never --image=curlimages/curl -- \
  curl -s -m6 -o /dev/null -w "%{http_code}\n" http://laravel-app.laravel-app-prod.svc.cluster.loca
l
```

4.6 Backups (d  clenchement    la demande)

```
kubectl -n laravel create job --from=cronjob/mysql-backup backup-demo
kubectl -n laravel get pods | grep backup          # attendre Completed
kubectl -n laravel logs job/backup-demo            # logs du backup
```

4.7 HTTPS (cert-manager) + SSO (dex/oauth2-proxy)

```
kubectl -n laravel-app-prod get certificate        # laravel-app-tls Ready=True
kubectl get clusterissuer                          # selfsigned-issuer
kubectl -n auth get pods                           # dex + oauth2-proxy Running
# V  rifier la porte SSO (acc  s non authentifi      Prometheus -> 302 vers dex) :
ING=$(kubectl -n ingress-nginx get svc ingress-nginx-controller -o jsonpath='{.spec.clusterIP}')
curl -s -o /dev/null -w "%{http_code} -> %{redirect_url}\n" -H "Host: prometheus.kubequest.local" h
ttp://$ING/oauth2/start
```

4.8 D  ploiement depuis docker-compose (via kubectl/helm/kustomize — exigence de d  fense)

```
kubectl apply -f production-ready/                # manifestes du repo local
helm list -A                                       # releases helm install  es
kubectl -n argocd get applications                 # applications g  r  es par GitOps
```

5. Dépannage keynote (« tout peut arriver »)

Symptôme	Solution
Impossible de joindre node1	1) Console AWS : node1 Running ; 2) IP = 35.156.165.130 (Elastic, fixe) ; 3) <code>ssh -v node1</code> pour l'erreur ; 4) clé <code>~/.ssh/ec2-epitech</code> présente.
node2/3/4 en NotReady	Souvent éteints : les Start dans la console AWS, attendre 2-3 min que <code>kubectl get nodes</code> passe Ready.
L'app ne répond pas	<code>kubectl -n laravel-app-prod get pods</code> ; si pas Running : <code>kubectl -n laravel-app-prod describe pod <nom></code> ; forcer : <code>kubectl -n laravel-app-prod rollout restart deploy/laravel-app</code> .
Le port-forward se coupe	Normal, Ctrl+C puis relancer la commande.
Un pod en CrashLoop	<code>kubectl -n <ns> logs <pod> --previous</code> .
OPA bloque une opération légitime	Si Gatekeeper refuse un pod voulu : supprimer la contrainte <code>kubectl delete k8srequiredlabels pods-must-have-name -label</code> (à recréer après la démo).
Prometheus inaccessible (SSO)	port-forward direct du service (§3.4) pour contourner ; ou login <code>admin@kubquest.local</code> / password.
Retirer un composant ajouté aujourd'hui	Voir §6 (désinstallation).
kubectl « connection refused »	Vous n'êtes pas sur node1 ; faire <code>ssh node1</code> d'abord.

Auto-diagnostic rapide (une commande)

```
ssh node1 'kubectl get nodes; echo ---; kubectl get pods -A | grep -vE "Running|Completed" ; echo "^^^ vide = tout est sain"'
```

6. À savoir & liste « surtout NE PAS faire »

- ❌ **Ne PAS « Sync » l'app ArgoCD `infra-security`** ! Elle déploierait l'ancien webhook OPA (`failurePolicy: Fail`, cluster-wide) qui **bloquerait instantanément tout le cluster** (plus aucun pod créable). On l'a remplacé par **Gatekeeper**.
- ❌ **Ne PAS faire de Sync global ArgoCD** : les 6 apps infra en `OutOfSync` sont normales (cert-manager etc. installés à la main).
- ⚠️ L'ancien namespace `laravel` a été **scalé à 0** (il était en CrashLoop permanent). L'app utilisée est `laravel-app-prod` (`larapp.kubquest.local`).
- ⚠️ **HTTPS public / Let's Encrypt non fait** : l'ingress n'est pas exposé publiquement (NodePort bloqué par le SG). On utilise un **certificat auto-signé**, démo via port-forward.
- ⚠️ **Dernier code (correctifs sécurité) non déployé sur le cluster** : image `imagePullPolicy: Never` + pas de registre + pas d'accès SSH à node2 pour distribuer l'image → rebuild/redeploy impossible. Correctifs dans PR #5/#6 (à mentionner : audit sécurité + correctifs CVE, en attente de merge).

Désinstallation / rollback des composants ajoutés aujourd'hui

```
# SSO
kubectl delete ns auth
kubectl -n monitoring annotate ingress prometheus-ingress nginx.ingress.kubernetes.io/auth-url- nginx.ingress.kubernetes.io/auth-signin-
# OPA
kubectl delete k8srequiredlabels pods-must-have-name-label; kubectl delete constrainttemplate k8srequiredlabels
kubectl delete -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.17.1/deploy/gatekeeper.yaml
# cert-manager
kubectl delete clusterissuer selfsigned-issuer
# NetworkPolicy
kubectl -n laravel-app-prod delete netpol laravel-app-isolation
# ingress monitoring
kubectl -n monitoring delete ingress grafana-ingress prometheus-ingress
```

7. Récapitulatif des identifiants

Service	Adresse (après port-forward)	Login	Mot de passe / token
ArgoCD	http://localhost:8081	admin	4SULul0BxohZNskt
Grafana	http://localhost:3000	admin	3d5vp6v7xYxx9IL5bFTlFGTxTwINpLpM1bwSnrHd
Dashboard	https://localhost:8443	(token)	kubectl -n kubernetes-dashboard create token admin-user
SSO / Prometheus	via ingress	admin@kuberequest.local	password
Application	http://localhost:8080	aucun	—